ASSIGNMENT 4 – CO4

ROBUST DATA PROCESSING WITH FILES, EXCEPTIONS, MODULES AND PACKAGES

Student Name: M SANTHOSH KUMAR

Register Number: 192412268

Task 1 – File-Processing Design Review (6 Marks)

1.1 Diagnose the Script (2 Marks)

Deficiency	What can go wrong in the hostel office?
Hard-coded absolute path	The script works only on the office computer with that exact folder structure; it fails on another computer or after files are moved.
File is never closed explicitly	The file handle remains open until cleanup; repeated runs can waste resources or interfere with file operations.
Reads the entire file with read()	Memory usage grows with the full file size; large files may consume excessive memory.
Bare except: pass	It hides every error, including programming mistakes, and gives no explanation for rejected records.
int(parts[2]) rejects paise	Values such as 1250.50 raise ValueError or are incorrectly handled; currency needs decimal-safe parsing.
Repeated string concatenation	Repeated report = report + ... can copy accumulated strings many times, increasing processing cost.
Trailing empty line	A final newline may produce an empty record; parts[2] then raises IndexError.
No rejected-line log	Invalid billing records disappear silently, so staff cannot correct records or audit totals.
No field-count / data validation	Malformed records or invalid amounts can be included incorrectly or cause exceptions.

1.2 Repaired Design (3 Marks)

```
from decimal import Decimal, InvalidOperation

def total_mess_bills(filename):
    total = Decimal("0.00")
    report_lines = []
    rejected_lines = []

    try:
        with open(filename, "r", encoding="utf-8") as f:
            for line_number, raw_line in enumerate(f, start=1):
                line = raw_line.strip()
                if not line:
                    continue

                parts = [part.strip() for part in line.split(",")]
                if len(parts) != 3:
                    rejected_lines.append((line_number, raw_line.rstrip("\\n"),
```

```
                                        "expected 3 comma-separated fields"))
                    continue

                register_no, name, amount_text = parts
                if not register_no or not name:
                    rejected_lines.append((line_number, raw_line.rstrip("\\n"),
                                           "register number or name is empty"))
                    continue

                try:
                    amount = Decimal(amount_text)
                    if not amount.is_finite() or amount < 0:
                        raise InvalidOperation
                    amount = amount.quantize(Decimal("0.01"))
                except InvalidOperation:
                    rejected_lines.append((line_number, raw_line.rstrip("\\n"),
                                           "invalid non-negative currency amount"))
                    continue

                total += amount
                report_lines.append(f"{name}: {amount:.2f}")

    except FileNotFoundError:
        raise FileNotFoundError(f"Mess bill file not found: {filename}")
    except PermissionError:
        raise PermissionError(f"Permission denied while reading: {filename}")

    return total, report_lines, rejected_lines


if __name__ == "__main__":
    filename = "mess_oct.txt"
    try:
        total, report, rejected = total_mess_bills(filename)
        print("\\n".join(report))
        print(f"Total: {total:.2f}")
        print("Rejected records:")
        for line_no, content, reason in rejected:
            print(f"Line {line_no}: {content!r} — {reason}")
    except (FileNotFoundError, PermissionError) as exc:
        print(f"Unable to process file: {exc}")
```

1.3 Design Defence (≤100 words) (1 Mark)

The repaired program processes one line at a time, so it does not need to store all 50,000 lines in memory. The with statement closes the file even if an error occurs. Decimal preserves currency values such as 1250.50. Specific validation checks reject malformed records without stopping the whole run, and each rejected line is recorded with its line number and reason. Building report entries in a list avoids repeated string concatenation. Passing the filename as a parameter makes the function reusable and easier to test.

Alternative Concept and Complexity Analysis

Both approaches take O(n) time for n input characters/records because each record must be examined. Using read() or readlines() requires O(N) additional memory for the complete file (N = file size). Iterating line by line needs O(L) input-buffer memory for the longest line L, plus O(R) for the report and rejected-line collections retained by this particular implementation. If the report is streamed to an output file instead, working memory can approach O(L) apart from error records that are intentionally retained.

Task 2 – From Requirement to File-Processing Design (8 Marks)

2.1 Model the Data (2 Marks)

Input: one daily UTF-8 text/CSV file per classroom or day. Each valid line has exactly three comma-separated fields: register_number,date,status. The status must be P or A. The month's date range and the expected daily files must be specified by the caller.

- Counts dictionary: register_number → {'present': integer, 'absent': integer}.
- Damaged-line log: tuples containing file name, line number, original line and reason.
- Attendance percentage: present / (present + absent) × 100. Students with no valid records need an explicit policy; this draft reports them as 0% and flags them as having no valid records.
- Output 1, attendance_summary.csv: register_number,present,absent,valid_days,percentage,below_75_percent.
- Output 2, damaged_lines.csv: file_name,line_number,raw_line,error_reason.

2.2 File Operations (2 Marks)

Operation	Choice	Reason
Read daily inputs	open(path, 'r', encoding='utf-8') and file iteration	UTF-8 handles ordinary text predictably; iteration is memory-efficient.
Write fresh reports	open(path, 'w', encoding='utf-8', newline='')	Replaces the previous report so stale rows are not retained.
Append persistent audit log	open(path, 'a', encoding='utf-8')	Use only when preserving logs across runs is required.
Read a line	for line in file	Maintains a line-by-line stream and supports line numbers.
Write CSV rows	csv.writer(...).writerow(...)	Correctly quotes commas and other special characters.

2.3 Monthly Processing Pseudocode (3 Marks)

```
INPUT: folder containing the month's daily attendance files
INITIALISE counts as an empty dictionary
INITIALISE damaged_records as an empty list

FOR each file path returned by pathlib.Path(folder).glob("*.csv"):
    OPEN file using UTF-8
    FOR each line with line_number starting at 1:
        IF line is blank: continue
        SPLIT line by comma and strip fields
        IF field count is not 3:
            append (filename, line_number, raw line, "wrong field count")
            continue
        READ register_number, date, status
        IF register_number/date is empty OR status not in {"P", "A"}:
            append a descriptive damaged record
            continue
        IF date is outside the requested month:
            append or skip according to the stated policy
            continue
        IF register_number not in counts:
            counts[register_number] = {present: 0, absent: 0}
        IF status == "P": increment present
        ELSE: increment absent
    CLOSE file automatically

WRITE attendance_summary.csv
FOR each student in counts:
    valid_days = present + absent
    percentage = 100 * present / valid_days if valid_days > 0 else 0
```

```
    write the summary row and whether percentage < 75

WRITE damaged_lines.csv with filename, line number, raw line and reason
```

2.4 Trace (1 Mark)

```
Input file: day01.csv
192311001,2026-10-08,P
192311002,2026-10-08,A
192311003,2026-10-08
192311001,2026-10-08,P

Trace:
Line 1 -> valid P -> 192311001: present=1, absent=0
Line 2 -> valid A -> 192311002: present=0, absent=1
Line 3 -> only 2 fields -> damaged log records day01.csv, line 3
Line 4 -> valid P -> 192311001: present=2, absent=0

Summary:
192311001: 2/2 = 100.00%
192311002: 0/1 = 0.00%
Damaged: (day01.csv, 3, "192311003,2026-10-08", "expected 3 fields")
```

Alternative Concept and Complexity Analysis

If the files are re-read separately for every student, the time is O(s·f·l), where s is the number of students, f the number of files and l the average number of lines per file. A single pass with a dictionary takes O(f·l) average time, assuming average O(1) dictionary operations. The dictionary uses O(s) space, while the damaged-line log uses O(d) space for d rejected lines. Re-reading can use less accumulated state but causes much more file I/O.

Task 3 – Design the Error Handling (9 Marks)

3.1 Failure Table (3 Marks)

Failure	Likely exception / signal	Handling
Folder or file does not exist	FileNotFoundError	Report the path; continue to other files where possible.
Insufficient permissions	PermissionError	Log the failure and continue with other files.
Wrong text encoding / invalid byte sequence	UnicodeDecodeError	Log file-level decoding error; do not silently treat as valid data.
Missing fields	Custom InvalidRecord; could otherwise cause IndexError	Reject that line with file and line number.
Invalid status such as L	Custom InvalidRecord or ValueError	Reject the line and preserve the reason.
Invalid date/number conversion	ValueError	Reject line or report invalid configuration.
Empty file	No exception by default	Return zero counts and optionally log a warning.
Path points to a directory	IsADirectoryError (platform dependent)	Treat as a file-level failure.
Disk full or write failure	OSError / subclass	Log/report the write failure; do not claim report succeeded.
Unexpected programming error	Other exception	Allow it to surface or log with traceback; avoid blanket pass.

3.2 Custom Exception (2 Marks)

```
class InvalidRecord(Exception):
    """A line does not match the attendance record specification."""
```

```
def parse_line(line):
    parts = [part.strip() for part in line.strip().split(",")]
    if len(parts) != 3:
        raise InvalidRecord(f"expected 3 fields, found {len(parts)}")
    register_no, date_text, status = parts
    if not register_no or not date_text:
        raise InvalidRecord("register number and date must be non-empty")
    if status not in {"P", "A"}:
        raise InvalidRecord(f"invalid status {status!r}")
    return register_no, date_text, status
```

Raise InvalidRecord inside parse_line(), where the format rule is known. Catch it inside the per-line loop in summarise(), where the program can record the line number and continue. Catch file-level exceptions around the open/read operation, because a missing or unreadable file is different from a single damaged line.

3.3 Complete try/except/else/finally and Completed Code (3 Marks)

```
import os
import tempfile


class InvalidRecord(Exception):
    """Raised when a line is not in the expected format."""


def parse_line(line):
    parts = line.strip().split(",")
    if len(parts) != 3:
        raise InvalidRecord(f"expected 3 fields, found {len(parts)}")
    register_no, date_text, status = (part.strip() for part in parts)
    if not register_no or not date_text:
        raise InvalidRecord("register number and date must be non-empty")
    if status not in {"P", "A"}:
        raise InvalidRecord(f"invalid status {status!r}")
    return register_no, date_text, status


def summarise(path):
    present, absent, errors = 0, 0, []
    try:
        with open(path, "r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                if not line.strip():
                    continue
                try:
                    _register_no, _date_text, status = parse_line(line)
                except InvalidRecord as exc:
                    errors.append((line_number, str(exc)))
                    continue
                else:
                    if status == "P":
                        present += 1
                    else:
                        absent += 1
    except FileNotFoundError as exc:
        raise FileNotFoundError(f"attendance file not found: {path}") from exc
    except PermissionError as exc:
        raise PermissionError(f"permission denied: {path}") from exc
    except UnicodeDecodeError as exc:
        raise ValueError(f"file is not valid UTF-8: {path}") from exc
    finally:
        # The with statement closes the file if it was successfully opened.
        # This finally block is useful for cleanup/diagnostics, not manual close.
```

```
        pass
    return present, absent, errors


data = ("192311001,2026-10-08,P\\n192311002,2026-10-08,A\\n"
        "192311003,2026-10-08\\n192311004,2026-10-08,L\\n")
with tempfile.NamedTemporaryFile("w", suffix=".csv", delete=False,
                                 encoding="utf-8") as tmp:
    tmp.write(data)
    temp_path = tmp.name

try:
    print(summarise(temp_path))
finally:
    os.remove(temp_path)

# Missing-file experiment:
try:
    summarise("this_file_does_not_exist.csv")
except FileNotFoundError as exc:
    print(exc)
```

Expected result for the supplied temporary file: (1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")]). The missing-file experiment prints a clear message instead of crashing without context.

3.4 Misuse Experiment (1 Mark)

```
# Deliberately incorrect experiment – run separately, then remove/fix it.
try:
    total_present = 0
    total_presnt += 1  # Typing error: total_presnt was never defined.
except Exception:
    pass

print("Program continues, but the NameError is hidden.")
```

The misspelled variable raises NameError. The blanket except Exception: pass catches it and discards the diagnostic, so the program appears to continue even though the calculation is wrong. This is dangerous because it hides both expected and unexpected failures.

Alternative Concept and Complexity Analysis

With exception-driven validation, valid records follow the normal path and exceptions are created only for bad records. This is often convenient when malformed lines are rare, although raising exceptions has overhead. With look-before-you-leap checks, the code checks field count and status using if statements before parsing or counting. That performs predictable conditional checks on every line and may be preferable when malformed lines are common. Both approaches are O(1) per line and O(n) for n lines overall; the practical constant factors depend on the language runtime and the fraction of bad records. Measure both on the same data rather than assuming one is faster.

Task 4 – File and Exception Forensics (7 Marks)

4.1 Locate the Defects (2 Marks)

- Opening returns.log with mode 'w' truncates the file every day, so earlier days are lost.
- The loop writes each book without adding a newline, so records are concatenated together.
- If write() fails before log.close(), the file may remain open until cleanup; buffered data may not be flushed reliably.
- The input is opened with open(day_file).read() without a with statement, so the input handle is not explicitly closed.
- Only FileNotFoundError is caught, and it prints 'missing' without preserving a timestamp, path, or diagnostic in an error log.
- Splitting on newline can create a final empty string; blank records should be skipped.

4.2 Test That Demonstrates the Defects (2 Marks)

```
from pathlib import Path
from tempfile import TemporaryDirectory

def save_returns_bad(books, log_path):
    log = open(log_path, "w", encoding="utf-8")
    for book in books:
        log.write(book)
    log.close()

def process_bad(day_file, log_path):
    try:
        books = open(day_file, encoding="utf-8").read().split("\\n")
        save_returns_bad(books, log_path)
    except FileNotFoundError:
        print("missing")

with TemporaryDirectory() as folder:
    folder = Path(folder)
    log_path = folder / "returns.log"
    for day, books in [
        ("day1.txt", ["Book A", "Book B"]),
        ("day2.txt", ["Book C"]),
        ("day3.txt", ["Book D", "Book E"]),
    ]:
        day_file = folder / day
        day_file.write_text("\\n".join(books), encoding="utf-8")
        process_bad(day_file, log_path)
        print(f"After {day}: {log_path.read_text(encoding='utf-8')!r}")

# The last printed log contains only day 3's records, joined without newlines.
```

4.3 Repaired Program (2 Marks)

```
from pathlib import Path
from datetime import datetime

def save_returns(books, log_path="returns.log", error_log="returns_errors.log"):
    try:
        with open(log_path, "a", encoding="utf-8") as log:
            for book in books:
                book = book.strip()
                if book:
                    log.write(book + "\\n")
    except OSError as exc:
        with open(error_log, "a", encoding="utf-8") as errors:
            errors.write(f"{datetime.now().isoformat()} write failure {log_path}: {exc}\\n")
        raise

def process(day_file, log_path="returns.log", error_log="returns_errors.log"):
    try:
        with open(day_file, "r", encoding="utf-8") as source:
            books = [line.strip() for line in source if line.strip()]
        save_returns(books, log_path, error_log)
    except (FileNotFoundError, PermissionError, UnicodeDecodeError, OSError) as exc:
        with open(error_log, "a", encoding="utf-8") as errors:
            errors.write(f"{datetime.now().isoformat()} {day_file}: {type(exc).__name__}: {exc}\\n")
        print(f"Could not process {day_file}: {exc}")
```

4.4 Prevention Rules (1 Mark)

1. Use with open(...) for every file so handles are closed even when exceptions occur.
2. Choose the file mode intentionally: 'a' to preserve prior log entries, 'w' only when replacing the whole file is intended; write one record per line.

Alternative Concept and Complexity Analysis

Manual handling can use try/finally: open the file, perform operations inside try, and close it in finally if the handle was successfully created. Both manual try/finally and with have O(n) time for n written characters and O(1) additional working space (excluding buffered I/O). Complexity does not change. With statements reliably close each successfully opened file. Manual code that forgets to close can leave one or more files open; a correct try/finally version closes every handle it successfully opened.

Task 5 – The Same Utility Organised Four Ways (7 Marks)

5.1 Four Organisations (3 Marks)

(a) One script

```
# attendance_all.py
def read_records(paths):
    records, errors = [], []
    for path in paths:
        try:
            with open(path, encoding="utf-8") as f:
                for line_no, line in enumerate(f, 1):
                    if not line.strip():
                        continue
                    parts = [p.strip() for p in line.split(",")]
                    if len(parts) != 3 or parts[2] not in {"P", "A"}:
                        errors.append((path, line_no, line.rstrip(), "invalid record"))
                    else:
                        records.append(tuple(parts))
        except OSError as exc:
            errors.append((str(path), 0, "", str(exc)))
    return records, errors

def percentage(records):
    counts = {}
    for reg, date, status in records:
        counts.setdefault(reg, [0, 0])
        counts[reg][0 if status == "P" else 1] += 1
    return {reg: (p / (p + a) * 100 if p + a else 0)
            for reg, (p, a) in counts.items()}

def write_report(results, path="summary.txt"):
    with open(path, "w", encoding="utf-8") as f:
        for reg, pct in sorted(results.items()):
            f.write(f"{reg},{pct:.2f}%\\n")

if __name__ == "__main__":
    records, errors = read_records(["day1.csv", "day2.csv"])
    write_report(percentage(records))
    print("Errors:", errors)
```

(b) One utility module and one main script

```
# attendance_utils.py
def percentage(records):
    counts = {}
    for reg, date, status in records:
        counts.setdefault(reg, [0, 0])
        counts[reg][0 if status == "P" else 1] += 1
    return {reg: (p / (p + a) * 100 if p + a else 0)
            for reg, (p, a) in counts.items()}

# main.py
```

```
from attendance_utils import percentage

def main():
    records = [("192311001", "2026-10-08", "P"),
               ("192311001", "2026-10-09", "A")]
    print(percentage(records))

if __name__ == "__main__":
    main()
```

(c) Three modules in one folder

```
# attendance_reader.py
def read_records(paths):
    records, errors = [], []
    for path in paths:
        try:
            with open(path, encoding="utf-8") as f:
                for line_no, line in enumerate(f, 1):
                    if not line.strip():
                        continue
                    parts = [x.strip() for x in line.split(",")]
                    if len(parts) == 3 and parts[2] in {"P", "A"}:
                        records.append(tuple(parts))
                    else:
                        errors.append((path, line_no, line.rstrip(), "invalid record"))
        except OSError as exc:
            errors.append((path, 0, "", str(exc)))
    return records, errors

# attendance_stats.py
def percentage(records):
    counts = {}
    for reg, date, status in records:
        counts.setdefault(reg, [0, 0])
        counts[reg][0 if status == "P" else 1] += 1
    return {reg: (p / (p + a) * 100 if p + a else 0)
            for reg, (p, a) in counts.items()}

# attendance_report.py
def write_report(results, path="summary.txt"):
    with open(path, "w", encoding="utf-8") as f:
        for reg, pct in sorted(results.items()):
            f.write(f"{reg},{pct:.2f}%\\n")

# main.py
from attendance_reader import read_records
from attendance_stats import percentage
from attendance_report import write_report

if __name__ == "__main__":
    records, errors = read_records(["day1.csv"])
    write_report(percentage(records))
    print(errors)
```

(d) Package named attendance/

```
attendance/
    __init__.py
    reader.py
    stats.py
    report.py
main.py

# attendance/__init__.py
```

```
"""Tools for processing attendance records."""
from .reader import read_records
from .stats import percentage
from .report import write_report

# attendance/reader.py
def read_records(paths):
    records, errors = [], []
    for path in paths:
        try:
            with open(path, encoding="utf-8") as f:
                for line_no, line in enumerate(f, 1):
                    if not line.strip():
                        continue
                    parts = [x.strip() for x in line.split(",")]
                    if len(parts) == 3 and parts[2] in {"P", "A"}:
                        records.append(tuple(parts))
                    else:
                        errors.append((path, line_no, line.rstrip(), "invalid record"))
        except OSError as exc:
            errors.append((path, 0, "", str(exc)))
    return records, errors

# attendance/stats.py
def percentage(records):
    counts = {}
    for reg, date, status in records:
        counts.setdefault(reg, [0, 0])
        counts[reg][0 if status == "P" else 1] += 1
    return {reg: (p / (p + a) * 100 if p + a else 0)
            for reg, (p, a) in counts.items()}

# attendance/report.py
def write_report(results, path="summary.txt"):
    with open(path, "w", encoding="utf-8") as f:
        for reg, pct in sorted(results.items()):
            f.write(f"{reg},{pct:.2f}%\\n")

# main.py
from attendance import read_records, percentage, write_report

def main():
    records, errors = read_records(["day1.csv", "day2.csv"])
    write_report(percentage(records))
    print("Errors:", errors)

if __name__ == "__main__":
    main()
```

5.2 Add a Three-Consecutive-Day Absence Report (2 Marks)

A function can examine each student's status by date, sort dates, and identify any run of three consecutive calendar days marked A. The function belongs in stats.py (or attendance_stats.py for the three-module layout); the output writer belongs in report.py (or attendance_report.py).

```
from datetime import date, timedelta

def absent_three_consecutive_days(records):
    by_student = {}
    for reg, date_text, status in records:
        try:
            day = date.fromisoformat(date_text)
        except ValueError:
            continue
```

```
        by_student.setdefault(reg, {})[day] = status

    result = set()
    for reg, status_by_day in by_student.items():
        for day, status in status_by_day.items():
            if (status == "A"
                and status_by_day.get(day + timedelta(days=1)) == "A"
                and status_by_day.get(day + timedelta(days=2)) == "A"):
                result.add(reg)
                break
    return sorted(result)
```

Organisation	Files to change for this requirement	Reason
One script	1 file	Add function and output logic in the same script.
Utility module + main	Usually 2 files	Add statistics function to utility module and call it from main.
Three modules	Usually 3 files	Add logic to stats module, output writer to report module, call from main.
Package	Usually 3 files	Add function to attendance/stats.py, report writer to attendance/report.py, call from main.py; __init__.py may also change if exporting the function.

5.3 Comparison and Import Differences (2 Marks)

Organisation	Readability	Reuse/testing	Naming conflicts
One script	Simple for small work; becomes crowded	Functions can be tested, but importing also loads the whole script unless guarded	Higher risk of duplicate names in a large file
One utility module	Separates reusable logic from main	Utility can be reused by other scripts	Module namespace helps
Three modules	Clear separation of reading, statistics and reports	Each module can be tested independently	Lower risk than a single namespace
Package	Best structure for a growing application	Reusable submodules and a stable package API	Qualified names reduce ambiguity

- import attendance.stats: binds the package name attendance; access the module using attendance.stats.percentage(...).
- from attendance import stats: binds the stats submodule name directly; call stats.percentage(...).
- from attendance.stats import percentage: binds the function name percentage directly; call percentage(...). This is concise but can create a naming conflict if another imported function has the same name.

Alternative Concept and Complexity Analysis

Keeping all functions in one script is workable for a small utility. As the code grows, finding and changing a function becomes harder because responsibilities are mixed. A package adds a one-time import cost when modules are first loaded; Python caches imported modules in sys.modules, so repeated imports normally reuse the loaded module. The package consumes memory for module objects and definitions, generally O(M) for M loaded module content, but this does not change the core attendance algorithm's complexity. The main benefit is maintainability and testability, not a guaranteed runtime improvement.

Task 6 – Design Challenge: College Fest Registration Desk (13 Marks)

Stage 1 – Interpret the Problem (2 Marks)

Input: five department text files. Each line is register number, name, department, e-mail, semicolon-separated events.

Outputs: clean master CSV, one participant file per event, summary report and error log.

- Each non-empty line must have exactly five comma-separated fields.
- Register number, name, department and e-mail must be non-empty.
- E-mail validation uses simple string checks: one '@', non-empty local/domain parts, a dot in the domain, and no spaces. This is a practical basic check, not full RFC validation.
- Events are split on semicolons, stripped, empty entries removed and duplicates within a row removed.
- Duplicate registrations are identified by register number. Keep the first valid registration and log later duplicates; this policy is explicit and testable.
- Missing or unreadable department files must be logged, and remaining files must still be processed.
- A damaged line or invalid e-mail must not stop processing of other lines.

Stage 2 – Processing Flow (2 Marks)

```
START
  |
  v
List expected department files
  |
  v
For each file: try to open with UTF-8
  |-- open/read failure --> append file-level error --> next file
  |
  v
Read one line and track line number
  |
  v
Split fields and validate count + required fields + e-mail
  |-- invalid --> append error log row --> next line
  |
  v
Register number already in seen set?
  |-- yes --> log duplicate --> next line
  |
  v
Add register number to seen set; add record to master list
  |
  v
For each chosen event: add participant to event list
  |
  v
Write master CSV, event participant files, summary and error log
  |
  v
END
```

Stage 3 – Critical Design Decisions (2 Marks)

- Duplicate key: normalized register number (strip whitespace and compare case-insensitively). Use a set for average O(1) membership tests.
- E-mail validation: reject spaces; split on '@' and require exactly two non-empty parts; require a dot in the domain and reject a domain starting/ending with a dot. This catches common mistakes but does not prove that an address exists.
- Record-level errors are caught around each line; file-level OSError and UnicodeDecodeError are caught around each file. Output-write failures should be reported and must not be described as successful output.

- Use csv.writer for comma-separated outputs so names and fields containing commas are escaped correctly.

Stage 4 – Package Structure and Pseudocode (4 Marks)

```
fest_registration/
    __init__.py
    reader.py       # read department files and preserve source location
    validation.py   # parse fields, validate e-mail and events
    processor.py    # deduplicate, build master and event groups
    report.py       # write CSV files, summary and error log
main.py
inputs/
    department_1.csv
    department_2.csv
    department_3.csv
    department_4.csv
    department_5.csv
```

```
PSEUDOCODE
seen_register_numbers <- empty set
master_records <- empty list
event_participants <- empty dictionary
errors <- empty list

FOR each expected department file:
    TRY open file as UTF-8 text
    IF file cannot be opened:
        add filename and exception to errors
        CONTINUE to next file

    FOR each line with its line number:
        IF line is blank: CONTINUE
        TRY:
            parse exactly five fields
            validate required fields and e-mail
            normalize register number
            IF register number in seen_register_numbers:
                add duplicate-registration error
                CONTINUE
            split events on semicolons; trim and remove empty/duplicate events
        EXCEPT InvalidRecord:
            add filename, line number, raw line and reason to errors
            CONTINUE

        add register number to seen_register_numbers
        append validated record to master_records
        FOR each event:
            append record summary to event_participants[event]

WRITE master_records to master.csv
FOR each event:
    write event_participants[event] to a safe event-specific CSV filename
WRITE counts to summary.txt
WRITE all errors to errors.csv
```

Stage 5 – Implementation and Demonstration (3 Marks)

The following implementation is a compact runnable package equivalent in one file for easy testing. For the final repository, split the functions into the package structure shown above. It demonstrates valid records, a duplicate register number, an invalid e-mail, a damaged line and a missing department file.

```
import csv
from pathlib import Path
from tempfile import TemporaryDirectory
```

```
class InvalidRecord(Exception):
    pass

def validate_email(email):
    if " " in email or email.count("@") != 1:
        return False
    local, domain = email.split("@")
    return bool(local and domain and "." in domain and
                not domain.startswith(".") and not domain.endswith("."))

def parse_registration(line):
    fields = [x.strip() for x in line.rstrip("\\r\\n").split(",")]
    if len(fields) != 5:
        raise InvalidRecord(f"expected 5 fields, found {len(fields)}")
    reg, name, department, email, event_text = fields
    if not all([reg, name, department, email]):
        raise InvalidRecord("register number, name, department and email are required")
    if not validate_email(email):
        raise InvalidRecord(f"invalid email {email!r}")
    events = list(dict.fromkeys(e.strip() for e in event_text.split(";") if e.strip()))
    if not events:
        raise InvalidRecord("at least one event is required")
    return {"register_number": reg.casefold(), "name": name,
            "department": department, "email": email, "events": events}

def process_department_files(paths, output_dir):
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    seen, master, event_map, errors = set(), [], {}, []

    for path in paths:
        path = Path(path)
        try:
            with path.open("r", encoding="utf-8") as f:
                for line_no, raw in enumerate(f, start=1):
                    if not raw.strip():
                        continue
                    try:
                        record = parse_registration(raw)
                        key = record["register_number"]
                        if key in seen:
                            raise InvalidRecord(f"duplicate register number {key!r}")
                        seen.add(key)
                        record["source_file"] = path.name
                        master.append(record)
                        for event in record["events"]:
                            event_map.setdefault(event, []).append(record)
                    except InvalidRecord as exc:
                        errors.append([path.name, line_no, raw.rstrip(), str(exc)])
        except (OSError, UnicodeDecodeError) as exc:
            errors.append([path.name, 0, "", f"{type(exc).__name__}: {exc}"])

    with (output_dir / "master.csv").open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["register_number", "name", "department", "email", "events", "source_file"])
        for r in master:
            writer.writerow([r["register_number"], r["name"], r["department"],
                             r["email"], ";".join(r["events"]), r["source_file"]])

    for event, participants in event_map.items():
        # Use a simple safe filename for this example.
        safe_event = "".join(c if c.isalnum() or c in "-_" else "_" for c in event)
        with (output_dir / f"event_{safe_event}.csv").open("w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
```

```
            writer.writerow(["register_number", "name", "department", "email"])
            for r in participants:
                writer.writerow([r["register_number"], r["name"], r["department"], r["email"]])

    with (output_dir / "summary.txt").open("w", encoding="utf-8") as f:
        f.write(f"Valid unique registrations: {len(master)}\\n")
        f.write(f"Events: {len(event_map)}\\n")
        for event, participants in sorted(event_map.items()):
            f.write(f"{event}: {len(participants)} participants\\n")
        f.write(f"Rejected/missing-file records: {len(errors)}\\n")

    with (output_dir / "errors.csv").open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["file_name", "line_number", "raw_line", "reason"])
        writer.writerows(errors)

    return master, event_map, errors

if __name__ == "__main__":
    with TemporaryDirectory() as temp:
        base = Path(temp)
        (base / "dept1.csv").write_text(
            "192311001,Asha,ECE,asha@example.com,Robotics;Quiz\\n"
            "192311002,Ravi,CSE,ravi@example.com,Quiz\\n"
            "192311001,Asha Duplicate,ECE,asha2@example.com,Music\\n"
            "192311003,Meena,EEE,not-an-email,Robotics\\n"
            "damaged,line\\n", encoding="utf-8")
        (base / "dept2.csv").write_text(
            "192311004,Imran,IT,imran@example.org,Music;Robotics\\n", encoding="utf-8")
        expected_files = [base / "dept1.csv", base / "dept2.csv", base / "dept3_missing.csv"]
        master, event_map, errors = process_department_files(expected_files, base / "output")
        print("Unique valid registrations:", len(master))
        print("Event counts:", {k: len(v) for k, v in event_map.items()})
        print("Errors:")
        for error in errors:
            print(error)
        print("Output files:", sorted(p.name for p in (base / "output").iterdir()))
```

Expected demonstration summary: 3 unique valid registrations; events Robotics, Quiz and Music; errors include the duplicate registration, invalid e-mail, damaged line, and missing department file. Run the code to record the exact printed order and output filenames in your evidence screenshots.

Alternative Concept and Complexity Analysis

Searching a list of all previous register numbers for every new registration takes $O(n^2)$ time in the worst case, because each of n records may scan a list that grows to n. A set-based duplicate check takes O(n) average total time because membership and insertion are average O(1). Both approaches need O(n) space to retain unique records; the set-based version also uses O(u) space for u distinct keys. The set approach is usually preferable for larger inputs.

Final Submission Checklist

- Create repository CSA08-<RegisterNumber>-Assignment4 and ensure faculty can access it.
- Add Task01, Task02, Task03, Task04, Task05 and Task06 folders.
- Save each completed program as .py and include all sample input files.
- Run each program and capture genuine output screenshots.
- Add outputs such as attendance_summary.csv, damaged_lines.csv, returns.log, master.csv, event participant files, summary.txt and errors.csv where applicable.
- Add README.md with name, register number, Python version, run commands, brief results and repository structure.

- Add requirements.txt. These examples use only the Python standard library, so no third-party dependencies are required.
- Commit progressively, at least one commit per task.
- Paste the working GitHub URL on the cover page and submit it as required.
- Check all file paths and test the programs from a clean copy of the repository before submission.

Suggested README.md Template

```
# CSA08 – Assignment 4 (CO4)

- Name: YOUR NAME
- Register Number: YOUR REGISTER NUMBER
- Python Version: Python 3.x
- Dependencies: Python standard library only

## Tasks
- Task01: File-processing diagnosis and repair
- Task02: Monthly attendance processor design
- Task03: Custom exceptions and error handling
- Task04: Append-only return log and error logging
- Task05: Script/module/package organisation
- Task06: College fest registration processor

## Run
python Task01/task1.py
python Task02/task2.py
python Task03/task3.py
python Task04/task4.py
python Task05/main.py
python Task06/main.py

## Evidence
Add output screenshots, sample input/output files, and notes describing tests actually executed.
```